

Testing / CI Lab Report

Reece Galgana

CS 472-1001

Dr. Businge

Phase 1: Test Coverage

Task 1.1: Set Up Your Team Repository for Test Coverage Lab

- **Screenshot of Your Terminal**

Provide a screenshot that shows:

- The `test_coverage_lab` folder inside your repository.
- A successful build of the setup files (i.e., running `pytest` without errors).

```
(venv) MacBook-Air-44:cs472 xmastersteel$ ls
Group-2
(venv) MacBook-Air-44:cs472 xmastersteel$ cd Group-2/
(venv) MacBook-Air-44:Group-2 xmastersteel$ ls
data      repo_mining  test_coverage_lab
(venv) MacBook-Air-44:Group-2 xmastersteel$ cd test_coverage_lab/
(venv) MacBook-Air-44:test_coverage_lab xmastersteel$ ls
LICENSE      migrations      requirements.txt
README.md     models          test.db
app.py        pytest.ini      tests
instance     requirements-dev.txt  venv
(venv) MacBook-Air-44:test_coverage_lab xmastersteel$ pytest
===== test session starts =====
platform darwin -- Python 3.10.19, pytest-7.4.4, pluggy-1.6.0 -- /Users/xmastersteel/cs472/Group-2/test_coverage_lab/venv/bin/python3.10
cachedir: .pytest_cache
rootdir: /Users/xmastersteel/cs472/Group-2/test_coverage_lab
configfile: pytest.ini
plugins: cov-4.1.0
collected 2 items

tests/test_account.py::test_account_role_assignment PASSED [ 50%]
tests/test_account.py::test_invalid_role_assignment PASSED [100%]

----- coverage: platform darwin, python 3.10.19-final-0 -----
Name                               Stmts   Miss  Cover   Missing
-----
models/__init__.py                   7        0   100%
models/account.py                   53       22    58%   30, 34, 47-49, 53-55, 59-61, 65-69, 73, 77, 87, 91, 95-96
tests/__init__.py                     0        0   100%
tests/test_account.py               35        4    89%   27-30
TOTAL                               95       26    73%

===== 2 passed in 0.29s =====
(venv) MacBook-Air-44:test_coverage_lab xmastersteel$
```

- **Commit Link from Your Forked Repository**

Submit a commit link that includes:

- The `test_coverage_lab` folder added to your repository.
- Your commit history reflecting the setup process.

Link:

<https://github.com/Akigaii/Group-2/commit/96e3cd9249cee12c60e33cff9d897cd606142d1d>

Task 1.2: Working with Python Test Coverage

- A link to your Pull Request.

Link: <https://github.com/MamuzaD/Group-2/pull/22>

- A copy of your test case.

```
# =====  
# Test: Test Positive Deposit  
# Author: Reece Galgana  
# Date: 2025-02-11  
# Description: Verify that depositing a positive amount correctly increases the  
# balance.  
# =====  
def test_positive_deposit():  
    account = Account(name="Gorilla Sushi",email="gorillasushi@gmail.com",  
        role="user",balance=0)  
    # Depositing small positive integer increases balance accordingly.  
    account.deposit(1)  
    assert account.balance == 1  
  
    # Depositing large positive integer increases balance accordingly.  
    account.deposit(2 ** 32)  
    assert account.balance == (2 ** 32) + 1  
  
    # Depositing small positive float increases balance accordingly.  
    account.deposit(1.982)  
    assert account.balance == (2 ** 32) + 1 + 1.982  
  
    # Depositing small positive float increases balance accordingly.  
    account.deposit(2 ** 32.1)  
    assert account.balance == (2 ** 32) + 1 + 1.982 + (2 ** 32.1)
```

- A brief explanation of what your test does and why it is important.

My test covers depositing positive amounts into an Account object. By doing so, it increments the Account's "balance" attribute, which updates in the database accordingly.

My test extends for small integer deposits, large integer deposits, small float deposits, and large float deposits. These cover all data types for positive deposits, covering all potential cases. It's important for these scenarios to be accounted for to ensure reliability within one of the most common actions for a bank account.

In tandem with the other test cases, especially ones for the validity for negative/invalid deposits, it maintains the integrity of the program among various conditions.

Phase 2: Test-Driven Development (TDD)

- A link to your Pull Request (PR) for the TDD Lab:

Link: <https://github.com/MamuzaD/Group-2/pull/23>

- A copy of the test case you wrote.

```
def test_nonexistent_counter(self, client):  
    # Create a counter.  
    client.post('/counters/topsushi')  
    # Try to access a different, nonexistent counter, should return 404.  
    result = client.get('/counters/gorillasushi')  
    assert result.status_code == status.HTTP_404_NOT_FOUND
```

- A brief explanation of what your test case does and how it contributes to the Counter API.

This test case above returns a 404 NOT FOUND error when trying to access a nonexistent counter. By verifying this, it ensures that this edge case within the Counter API is accounted for. This improves the overall validity and integrity of the API.

- A summary of your **RED-GREEN-REFACTOR** process, including:

- The failing test (**RED** Phase).

```
def test_nonexistent_counter(self, client):  
    # Create a counter.  
    client.post('/counters/topsushi')  
    # Try to access a different, nonexistent counter, should return 404.  
    result = client.get('/counters/gorillasushi')  
    assert result.status_code == status.HTTP_404_NOT_FOUND
```

In accordance with the red aspect, the same unit test above will ultimately fail, because there is no overall functionality for it within counter.py.

- The implemented feature that made the test pass (**GREEN** Phase).

```
@app.route('/counters/<name>', methods=['GET'])  
def nonexistent_counter(name):  
  
    # Determines if counter exists.  
    counter_nonexistent = True
```

```

for counter in COUNTERS:
    if counter == name:
        counter_nonexistent = False

# Returns 404 if nonexistent, 200 if it does.
if counter_nonexistent:
    return jsonify({"error": f"Counter {name} is nonexistent"}),
        status.HTTP_404_NOT_FOUND
return jsonify({name: COUNTERS[name]}), status.HTTP_200_OK

```

This function within [counter.py](#) scans if the input counter name exists within the entire COUNTER array. If it does not, it returns an ERROR 404. This is utilized by the unit test and red phase above.

- **Any code improvements or refactoring you made (REFACTOR Phase).**

```

@app.route('/counters/<name>', methods=['GET'])
def nonexistent_counter(name):
    if not counter_exists(name):
        return jsonify({"error": f"Counter {name} is nonexistent"}),
            status.HTTP_404_NOT_FOUND
    return jsonify({name: COUNTERS[name]}), status.HTTP_200_OK

```

Instead of having a separate boolean function that separately iterates through COUNTERS, I consolidated it into the conditional statement. It also utilizes the helper function, `counter_exists()`. These changes would ultimately improve code performance and ease readability.

Phase 3: Continuous Integration

1. Link to your fork repository (GitHub URL):

Link: <https://github.com/Akigaii/Group-2>

2. Links to your three PRs in your fork (Pipeline, Testing, Debugging).

Pipeline PR: <https://github.com/Akigaii/Group-2/pull/2>

Testing PR: <https://github.com/Akigaii/Group-2/pull/3>

Debugging PR: <https://github.com/Akigaii/Group-2/pull/4>

3. Coverage Evidence (Testing PR):

Before (94%):

```
(venv) MacBook-Air-44:ci_lab xmastersteel$ pytest
===== test session starts =====
platform darwin -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0 -- /Users/xmastersteel/cs472/Group-2/ci_lab/venv/bin/python3.14
cachedir: .pytest_cache
rootdir: /Users/xmastersteel/cs472/Group-2/ci_lab
configfile: pytest.ini
plugins: cov-7.0.0
collected 22 items

tests/test_counter.py::TestCounterEndpoints::test_create_counter PASSED [ 4%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_duplicate_counter PASSED [ 9%]
tests/test_counter.py::TestCounterEndpoints::test_retrieve_existing_counter PASSED [ 13%]
tests/test_counter.py::TestCounterEndpoints::test_return_404_for_non_existent_counter PASSED [ 18%]
tests/test_counter.py::TestCounterEndpoints::test_increment_counter PASSED [ 22%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_updating_non_existent_counter PASSED [ 27%]
tests/test_counter.py::TestCounterEndpoints::test_delete_counter PASSED [ 31%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_deleting_non_existent_counter PASSED [ 36%]
tests/test_counter.py::TestCounterEndpoints::test_reset_all_counters PASSED [ 40%]
tests/test_counter.py::TestCounterEndpoints::test_list_all_counters PASSED [ 45%]
tests/test_counter.py::TestCounterEndpoints::test_handle_invalid_http_methods PASSED [ 50%]
tests/test_counter.py::TestCounterEndpoints::test_get_total_counters PASSED [ 54%]
tests/test_counter.py::TestCounterEndpoints::test_top_n_counters PASSED [ 59%]
tests/test_counter.py::TestCounterEndpoints::test_bottom_n_counters PASSED [ 63%]
tests/test_counter.py::TestCounterEndpoints::test_set_counter_to_value PASSED [ 68%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_negative_counter_values PASSED [ 72%]
tests/test_counter.py::TestCounterEndpoints::test_reset_single_counter PASSED [ 77%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_resetting_non_existent_counter PASSED [ 81%]
tests/test_counter.py::TestCounterEndpoints::test_get_total_number_of_counters PASSED [ 86%]
tests/test_counter.py::TestCounterEndpoints::test_counters_greater_than_threshold PASSED [ 90%]
tests/test_counter.py::TestCounterEndpoints::test_counters_less_than_threshold PASSED [ 95%]
tests/test_counter.py::TestCounterEndpoints::test_validate_counter_name PASSED [100%]

===== tests coverage =====
coverage: platform darwin, python 3.14.0-final-0

Name      Stmts  Miss  Cover   Missing
-----
src/__init__.py      4      0  100%
src/counter.py      87      5   94%   72, 86, 97, 100-101
src/status.py         6      6    0%   2-7
TOTAL              97     11   89%

===== 22 passed in 0.13s =====
```

After (97%):

```
(venv) MacBook-Air-44:ci_lab xmastersteel$ pytest
===== test session starts =====
platform darwin -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0 -- /Users/xmastersteel/cs472/Group-2/ci_lab/venv/bin/python3.14
cachedir: .pytest_cache
rootdir: /Users/xmastersteel/cs472/Group-2/ci_lab
configfile: pytest.ini
plugins: cov-7.0.0
collected 23 items

tests/test_counter.py::TestCounterEndpoints::test_create_counter PASSED [ 4%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_duplicate_counter PASSED [ 8%]
tests/test_counter.py::TestCounterEndpoints::test_retrieve_existing_counter PASSED [ 13%]
tests/test_counter.py::TestCounterEndpoints::test_return_404_for_non_existent_counter PASSED [ 17%]
tests/test_counter.py::TestCounterEndpoints::test_increment_counter PASSED [ 21%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_updating_non_existent_counter PASSED [ 26%]
tests/test_counter.py::TestCounterEndpoints::test_delete_counter PASSED [ 30%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_deleting_non_existent_counter PASSED [ 34%]
tests/test_counter.py::TestCounterEndpoints::test_reset_all_counters PASSED [ 39%]
tests/test_counter.py::TestCounterEndpoints::test_list_all_counters PASSED [ 43%]
tests/test_counter.py::TestCounterEndpoints::test_handle_invalid_http_methods PASSED [ 47%]
tests/test_counter.py::TestCounterEndpoints::test_get_total_counters PASSED [ 52%]
tests/test_counter.py::TestCounterEndpoints::test_top_n_counters PASSED [ 56%]
tests/test_counter.py::TestCounterEndpoints::test_bottom_n_counters PASSED [ 60%]
tests/test_counter.py::TestCounterEndpoints::test_set_counter_to_value PASSED [ 65%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_negative_counter_values PASSED [ 69%]
tests/test_counter.py::TestCounterEndpoints::test_reset_single_counter PASSED [ 73%]
tests/test_counter.py::TestCounterEndpoints::test_prevent_resetting_non_existent_counter PASSED [ 78%]
tests/test_counter.py::TestCounterEndpoints::test_get_total_number_of_counters PASSED [ 82%]
tests/test_counter.py::TestCounterEndpoints::test_counters_greater_than_threshold PASSED [ 86%]
tests/test_counter.py::TestCounterEndpoints::test_counters_less_than_threshold PASSED [ 91%]
tests/test_counter.py::TestCounterEndpoints::test_validate_counter_name PASSED [ 95%]
tests/test_counter.py::TestCounterEndpoints::test_no_counters_available PASSED [100%]

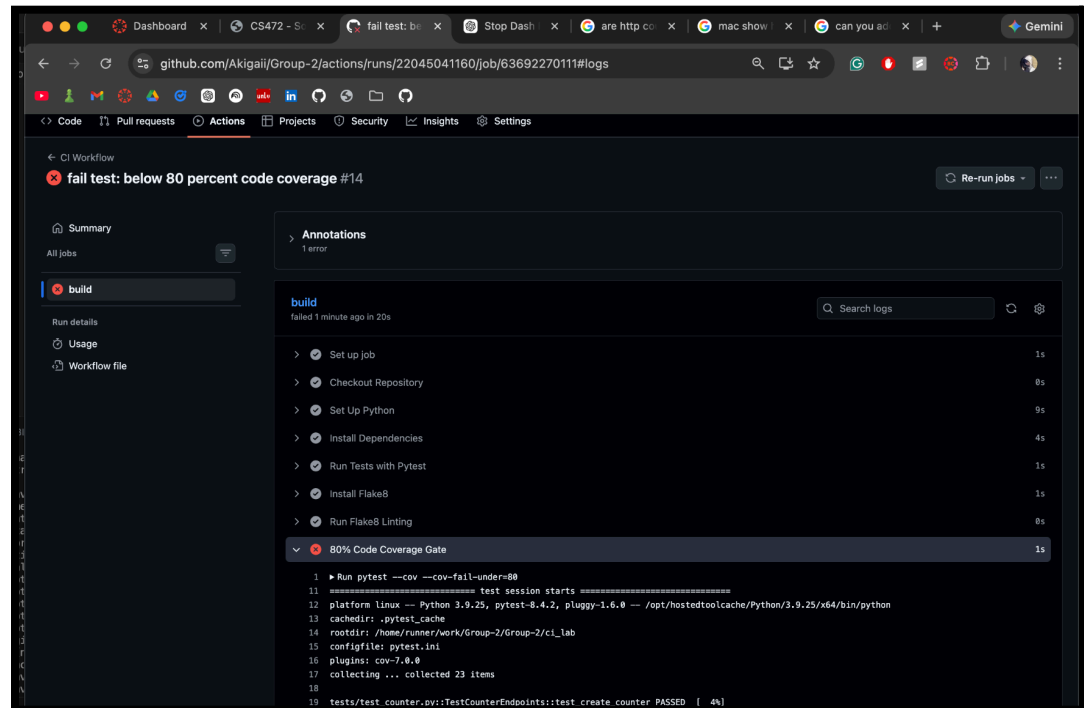
===== tests coverage =====
coverage: platform darwin, python 3.14.0-final-0

Name      Stmts  Miss  Cover   Missing
-----
src/__init__.py      4      0  100%
src/counter.py      87      3   97%   72, 86, 97
src/status.py         6      6    0%   2-7
TOTAL              97      9   91%

===== 23 passed in 0.12s =====
```

4. CI Log Evidence (Debugging PR):

Failing CI Log Snippet:



Fix Summary:

The enhancement I added to the CI workflow YAML was enforcing an 80% code coverage among all files. To force a failing test, I inserted several dummy print statements within a conditional, artificially inflating the number of lines unaccounted for.

Running and pushing after this expectedly yielded a failing test. Diagnosing via the Actions tab on GitHub, we witness that, indeed, the 80% code coverage job was the one that failed.

Refixing this, I simply removed the manually inserted lines and repushed. Afterward, all of the tests, including the 80% coverage gate, passed. All of this verifies this enhancement works as intended.

5. Code Excerpts:

New Test Function (Testing PR):

```
# =====
# Test: Prevent setting counter values with an invalid integer.
# Author: Reece Galgana
# Modification: Ensures a bad request code is returned upon invalid assignment.
```



```
# =====  
def test_no_counters_available(self, client):  
    client.post('/counters/test')  
    response = client.put('counters/test/set/this_is_not_an_integer')  
    assert response.status_code == HTTPStatus.BAD_REQUEST
```

CI Workflow YAML Enhancement (Pipeline PR):

```
-   name: 80% Code Coverage Gate  
  
    run: pytest --cov --cov-fail-under=80
```

6. A paragraph explaining what you learned about CI and automated quality checks.

This lab solidified the importance of Continuous Integration (CI) and automated quality checks, especially when working in real-life production environments. Humans are not perfect, so programmers are destined to make flaws/mistakes within their code. By enforcing these explicit guidelines, it prevents any undetected surprises when pushing programs, especially those with several team members. Additionally, this frequency and repetitions incentives active and clean collaboration among team members, ensuring that set standards are always met. These principles will most definitely be applied within our group project, assuring seamless integration among the different development teams.